Práctico 04 — Armado de la aeronave

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

11 de septiembre de 2026



Roadmap de la clase

Antecedentes

- Unidad VI — transformaciones
- Primitivas generadas: cubo, cilindro, cono
- Una matriz de modelo por objeto

Hoy

- Piezas que **dependen** de otras piezas
- **Deformar** una primitiva para que sirva
- Organizar el proyecto

Conectado a

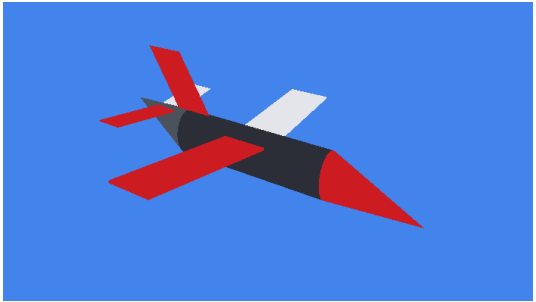
- Unidad VII — proyección
- Unidad VIII — caras ocultas
- Unidad IX — iluminación

Las cajas negras de hoy

- Sigue la **constante** en el vertex shader: es vista y proyección juntas.

Modelo poligonal de aeronave





La primera aeronave del proyecto será un modelo simplificado, aproximado mediante una composición de primitivas: **hay que crear esta composición** usando las primitivas desarrolladas hasta acá y agregando las que hagan falta.

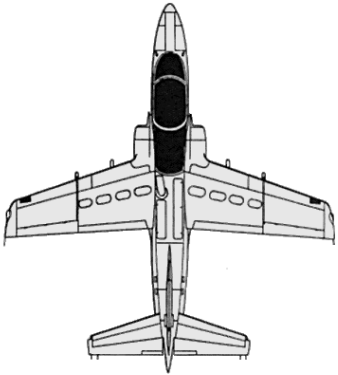
Actividad de aula — Definir la composición

1. **Fijar el sistema de referencia.** Sobre las tres vistas de la aeronave: marcar dónde va el **origen** del modelo y hacia dónde apunta cada eje.
2. **Despiece.** Para cada parte principal del avión —fuselaje, nariz, alas, empenajes—: elegir **qué primitiva** la aproxima, **qué medidas** tiene, y **qué transformaciones y en qué orden** hay que aplicarle.
3. **Contar.** ¿Cuántas **mallas distintas** hay que generar? ¿Y cuántas **matrices de modelo** por cuadro? ¿Por qué no coinciden?

Ayuda

Medidas **relativas al largo del fuselaje** ($L = 1$): no hace falta ningún dato real.

Actividad de aula — Definir la composición







Transformaciones aplicadas al modelo completo

Dependiendo de la composición del modelo propuesta, el mismo puede haber quedado organizado como:

Mallas: seis, una sola vez

Ej.: Dos conos, un cilindro, tres cubos.
Se generan al crear el modelo y no se vuelven a modificar.

Transformaciones: ocho, por cuadro

Una por primitiva.
Al reposicionar el modelo en la escena hay que actualizarlas.

La "escena" del práctico anterior tenía tres objetos **independientes**. En cambio, ahora, hay una *composición* de primitivas que forman **un solo objeto** y por lo tanto las transformaciones no son independientes.

Si quiero modificar la actitud del avión, ¿cómo hay que transformar las primitivas para reflejarla?

Hay que determinar una transformación única para todo el modelo que se aplique de manera jerárquica a los componentes del mismo.

Matriz de posición y orientación (*pose*)

Se usa una única matriz de transformación que describe los dos componentes que forman la "pose" de un objeto: posición y orientación.

La idea es aplicarla de manera jerárquica a todos los componentes que forman el modelo. Por ejemplo, para un alerón montado sobre el ala, que a su vez está montada sobre el avión:

$$M_{\text{mundo}}(\text{alerón}) = \underbrace{M_{\text{pose}}}_{\text{el avión en el mundo}} \cdot \underbrace{M_{\text{ala}}}_{\text{el ala en el avión}} \cdot \underbrace{M_{\text{alerón}}}_{\text{el alerón en el ala}}$$

Las matrices a la derecha de la pose describen cómo está armado el modelo y **no cambian con la pose**: su producto se calcula una sola vez y queda como la matriz *local* de la pieza. La jerarquía se reduce a dos niveles:

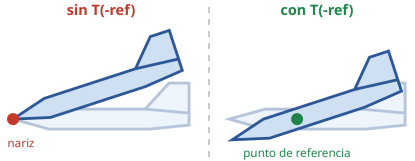
$$M_{\text{local}}(\text{alerón}) = M_{\text{ala}} \cdot M_{\text{alerón}} \quad \Longrightarrow \quad M_{\text{mundo}}(\text{pieza}) = \underbrace{M_{\text{pose}}}_{\text{cambia por cuadro}} \cdot \underbrace{M_{\text{local}}(\text{pieza})}_{\text{fija: se arma una vez}}$$

Punto de referencia de la pose

Los datos de la *pose* siempre están referidos a un punto particular del modelo.

Al aplicar la misma hay que hacer coincidir el sistema de referencia del modelo con el punto respecto al cuál están dados los datos de *pose*.

Para el caso de la aeronave: los datos de pose están referidos al centro de gravedad de la misma por lo tanto hay que aplicar la transformación de pose respecto a ese punto.



$$M_t = \underbrace{T(\text{posición}) \cdot R(\text{ángulos})}_{\text{pose}} \cdot \underbrace{T(-\text{ref})}_{\text{coincide el punto de referencia y el origen}}$$

En código

```
// --- init(): posición de la pieza en el modelo, una sola vez ------------
// Origen del modelo en la nariz; z+ va de la nariz a la cola.
glm::mat4 m_cuerpo = glm::mat4(1.0f);
m_cuerpo = glm::translate(m_cuerpo, glm::vec3(0.0f, 0.0f, z_cuerpo));
m_cuerpo = glm::rotate(m_cuerpo, glm::radians(-90.0f), eje_x);

// --- update(): aplicar pose. una cada cuadro ----------------------------
glm::mat4 m_pose = glm::mat4(1.0f);
m_pose = glm::translate(m_pose, pos_avion);
m_pose = glm::rotate(m_pose, cabeceo, eje_x);
m_pose = glm::translate(m_pose, -punto_ref);        // <-- pos ref to origin

// --- dibujar: acá se produce la composicion -----------------------------
for (const auto& pieza : piezas)
    draw(pieza.malla, m_pose * pieza.m_cuerpo);
```

rotate -90° es para orientar correctamente la pieza en el sistema de referencia del modelo (el fuselaje va sobre **Z** en el ejemplo propuesto).

Proponer una arquitectura de módulos

El primer modelo de la aeronave es una composición de primitivas ahora, pero más adelante puede que cambie a un modelo más refinado.

Debería estar encapsulado en una clase para poder abstraer la definición del modelo (primitivas vs modelo externo) y las transformaciones que hay que aplicarle.

¿En `main.cpp`?

Es donde está ahora. Funciona, pero no escala cuando se agreguen otros componentes

¿En `Mesh`?

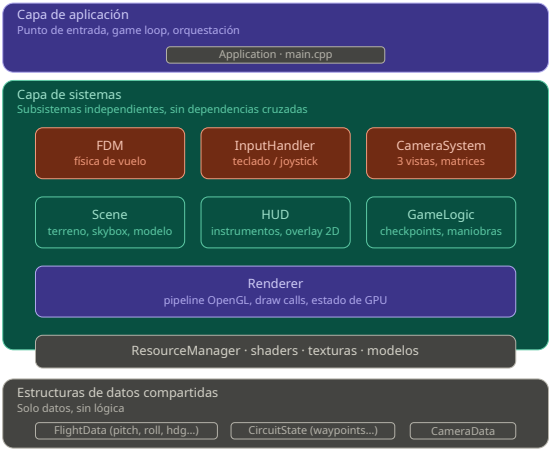
`Mesh` posee el VAO y carga bytes. No sabe qué configuración tiene el modelo.

¿En `primitives`?

Genera geometrías a partir de parámetros. No sabe dónde va esa geometría ni que modelo compone.

Hay que empezar a definir la arquitectura de los módulos que formarán el proyecto para no terminar con un `main.cpp` de 10000 líneas.

Proponer arquitectura de módulos



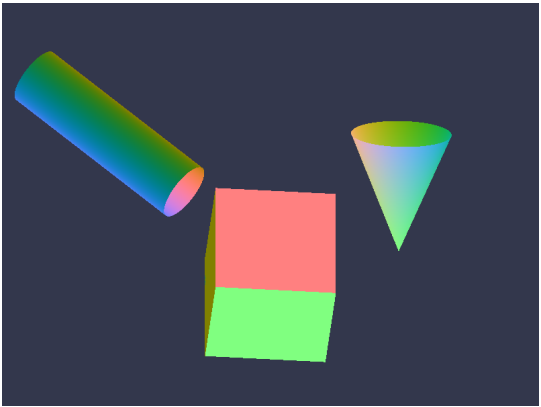
Responsabilidades de los módulos

Módulo	Responsabilidad	NO hace
`main / Application`	Inicializa todo, corre el game loop	Lógica de negocio
`ResourceManager`	Carga y cachea shaders, texturas y modelos por nombre	Nada gráfico directo
`Shader`	Compila, linkea y setea uniforms	Decidir qué se dibuja
`Mesh`	Posee VAO/VBO/EBO, cargar bytes	Saber qué representa
`primitives`	Genera geometría a partir de medidas	Saber dónde va
`Aircraft / Scene`	Qué modelos hay y dónde está cada una	Emitir draw calls
`Renderer`	Emite draw calls, gestiona estado de GPU	Física, lógica

¿Y lo nuevo que vaya surgiendo? — el criterio por funcionalidad

- **Input / física** — todo lo que toca hardware de entrada o el FDM.
- **Lógica / CG** — todo lo que implementa un requerimiento del programa o de computación gráfica.
- **Transversal** — orquestación y pipeline de GPU.
- **Infraestructura / datos** — recursos y estructuras compartidas, sin lógica.

Requisitos - Demo



Práctico 03 — `escena-primitivas` **coloreada usando las normales**

Estadísticas de las primitivas

	vértices	**índices**
cubo	24	36
cilindro	74	216
cono	38	108

Hay que tener en marcha este práctico para poder continuar al armado de la aeronave.

Taller: armar un modelo de una aeronave

Usando el despiece realizado en la actividad previa y el código del Práctico 03, hay que armar un modelo de la aeronave usando una malla poligonal.

1. **Plantear el fuselaje** usando las primitivas propuestas. En caso de no saber bien qué hacer, empezar usando un cilindro para la parte central del fuselaje.
2. **Agregar dos piezas más** —ej. un ala y la nariz, o las dos alas, o un ala y el empenaje vertical—. Cada una con **su matriz local**, armada desde la identidad, en coordenadas del avión.
3. **La prueba**. Construir una **matriz de pose** para rotar la aeronave y aplicarla al modelo obtenido hasta ahora. Todas las piezas que formen el modelo tienen que moverse juntas y rotar alrededor del punto de referencia.
4. Si queda tiempo agregar más piezas para completar la aeronave.

Práctico 04 — Armar modelo de la aeronave

1. Terminar el armado del avión usando el despiece propuesto propio
2. Prueba del punto 3 del taller (rotar sólo el avión para que suba / baje la nariz — cabeceo — rotación alrededor del eje de rolido)